Making Your Data Tell a Story

Matplotlib + Seaborn in Python

Tarun Gupta

THE PROBLEM

Default Charts Kill Good Analysis

- Gray backgrounds, tiny fonts, cluttered axes
- First impressions form before the numbers land
- Visualization is communication, not decoration

ARCHITECTURE

matplotlib vs seaborn: Layers, Not Competitors

- Seaborn is built on top of matplotlib
- Seaborn for the 80%: fast, good-looking defaults
- matplotlib for the 20%: pixel-level control

seaborn	matplotlib
high-level API	rendering engine

SETUP

One Block for Every Notebook

- `sns.set_theme(style='whitegrid', font_scale=1.2)`
- Remove top + right spines — they add no information
- Set figure DPI to `150` — default `72` is too low

NOTEBOOK SETUP BLOCK

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(
    style='whitegrid',
    font_scale=1.2
)
for ax in fig.axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
```

EXPLORATORY DATA ANALYSIS

5 Plot Types That Cover 90% of EDA

- `histplot` → distribution shape
- `scatterplot` → two-variable relationship
- `barplot` → categorical comparison
- `heatmap` → correlation structure
- `boxplot` → distribution across categories

KEY INSIGHT

Combining seaborn + matplotlib

- Every seaborn call returns a matplotlib object
- Use axes-level functions with explicit `ax=` targets
- Build a full dashboard in under 25 lines

```
fig, axes = plt.subplots(1, 2)

sns.histplot(
    df['age'], ax=axes[0]
)
sns.scatterplot(
    data=df, x='age',
    y='fare', ax=axes[1]
)
```

POLISH

Three Details That Make or Break a Chart

- Units on every axis — `'Fare (£)'` not `'Fare'`
- Consistent color assignment across figures
- Zero baseline on all bar charts — always

01 Label axes with units, always

02 Lock color palettes per category

03 Bar charts start at zero — no exceptions

OUTPUT

Saving Publication-Quality Output

- `dpi=300` — not the 72 default
- `bbox_inches='tight'` prevents edge cropping
- SVG and PDF for resolution-independent files

SAVE FOR PUBLICATION

```
fig.savefig(
    'chart.png',
    dpi=300,
    bbox_inches='tight'
)

# resolution-independent
fig.savefig('chart.svg')
fig.savefig('chart.pdf')
```

COMING UP

Tutorial 6: Statistical Testing

- scipy + statsmodels — the two libraries you need
- When to use which test (t-test, chi-square, ANOVA)
- Reporting results in plain English, not p-value soup

> *"Technical skill is necessary. But presentation is what gets heard in the room."*

FOLLOW FOR TUTORIAL 6

Your chart is done.
Is it telling the right story?

Follow **@breathofdatascience** — Tutorial 6 drops next week:

scipy + statsmodels · when to use which test · results in plain English

Save this · Share with your team